

# CS329e\_Group23\_FinalNotebook\_NoOutputs

April 27, 2026

## 1 Forecasting Airline Realized Volatility with HAR-RV, IV, OVX, and TOSI

### 1.1 Self-Contained Raw-Data Notebook

### 1.2 0. Project Goal

The goal of this notebook is to forecast the **variance risk premium (VRP)** – the difference between realized and implied volatility – and use that forecast to drive a vol trading strategy (buy/sell straddles) on airline stocks and the JETS ETF.

The notebook: - computes realized variance directly from raw Alpaca 5-minute bars - reframes the forecast target as **VRP**:  $y_{t+1} = \log(RV_{t+1}) - \log(IV_t)$  (both on daily-variance scale) - estimates walk-forward forecasts for **six** HAR-style feature sets: HAR-RV, HAR-RV+OVX, HAR-RV+OVX+TOSI (no IV), and HAR-RV+IV, HAR-RV+IV+OVX, HAR-RV+IV+OVX+TOSI - compares **OLS**, **Ridge**, **Random Forest**, and **XGBoost** - selects the trading threshold adaptively inside each walk-forward fold using a nested validation window (25th–75th percentile of  $|\text{predicted VRP}|$  on training data, evaluated on the validation sub-window, never on OOS data) - tracks the fraction of days traded per fold and warns when it falls below 20–30% - evaluates straddle Sharpe on the held-out test folds with no look-ahead

AAL is included now that the IV workbook reflects the correct U.S. security.

### 1.3 1. Setup

Import the same core packages used in the reference notebook, define shared colors and layout defaults, and keep all file access local to the raw data directory.

```
[ ]: # --- Imports ---
import warnings
from pathlib import Path

warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
from IPython.display import display
from plotly.subplots import make_subplots
import plotly.graph_objects as go
import plotly.io as pio
```

```

from scipy import stats
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import TimeSeriesSplit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from xgboost import XGBRegressor

pio.renderers.default = 'notebook_connected'
pd.options.display.float_format = '{:,.4f}'.format

# --- Constants ---
TICKERS = ['DAL', 'UAL', 'LUV', 'AAL', 'JETS']
WEEK_WINDOW = 5
MONTH_WINDOW = 22

WF_MIN_TRAIN_DAYS = 756
WF_TEST_DAYS = 63
WF_STEP_DAYS = 63

# Threshold grid: percentiles of |predicted VRP| on the pre-validation training
↳portion
THRESHOLD_PERCENTILES = list(range(25, 76, 5))
MIN_TRADES_FOR_SHARPE = 10 # minimum traded days in validation window to
↳score a threshold
TRADE_FRAC_WARN = 0.20 # warn if fewer than this fraction of test days
↳are traded

# ATM straddle cost factor: for  $R \sim N(0, \hat{\sigma}^2)$ ,  $E[|R|] = \hat{\sigma} \cdot \sqrt{2/\pi}$ .
# The fair ATM straddle price (unit notional, 1-day) is therefore  $\sqrt{2/\pi} \hat{\sigma}$ 
↳iv_daily_vol.
SQRT2_PI = np.sqrt(2 / np.pi)

RV_FEATURES = ['log_rv_daily', 'log_rv_weekly', 'log_rv_monthly']
# IV enters only as the current-period level: it is a market price, not a flow
↳quantity,
# so rolling averages obscure the contemporaneous signal and add spurious
↳collinearity.
IV_FEATURES = ['log_iv_daily']
OVX_FEATURES = ['log_ovx_daily', 'log_ovx_weekly', 'log_ovx_monthly']
TOSI_FEATURES = ['tosi_level', 'tosi_change']

# Six specs: two without IV (for comparing pure RV+macro signal vs IV-augmented)
FEATURE_SPECS = {
    'HAR-RV': RV_FEATURES,
    'HAR-RV+OVX': RV_FEATURES + OVX_FEATURES,

```

```

    'HAR-RV+OVX+TOSI': RV_FEATURES + OVX_FEATURES + TOSI_FEATURES,
    'HAR-RV+IV': RV_FEATURES + IV_FEATURES,
    'HAR-RV+IV+OVX': RV_FEATURES + IV_FEATURES + OVX_FEATURES,
    'HAR-RV+IV+OVX+TOSI': RV_FEATURES + IV_FEATURES + OVX_FEATURES +
    ↪TOSI_FEATURES,
}

MODEL_FAMILIES = ['OLS', 'Ridge', 'Random Forest', 'XGBoost']

AIRLINE_COLORS = {
    'DAL': '#0B6E4F',
    'UAL': '#3C91E6',
    'LUV': '#F4A259',
    'AAL': '#A1303A',
    'JETS': '#7A5195',
}

MODEL_COLORS = {
    'OLS': '#2D3047',
    'Ridge': '#0B6E4F',
    'Random Forest': '#F58518',
    'XGBoost': '#C75146',
}

SPEC_COLORS = {
    'HAR-RV': '#7A5195',
    'HAR-RV+OVX': '#F4A259',
    'HAR-RV+OVX+TOSI': '#2D3047',
    'HAR-RV+IV': '#0B6E4F',
    'HAR-RV+IV+OVX': '#C75146',
    'HAR-RV+IV+OVX+TOSI': '#3C91E6',
}

SHADED_PERIODS = [
    ('2022-02-01', '2022-08-31', 'Energy shock'),
    ('2024-05-01', '2026-02-15', 'Walk-forward OOS'),
]

BASE_LAYOUT = dict(
    template='plotly_white',
    paper_bgcolor='#F7F4ED',
    plot_bgcolor='FFFFFF',
    font=dict(family='Aptos, Segoe UI, sans-serif', size=13, color='#24323D'),
    colorway=['#0B6E4F', '#C75146', '#3C91E6', '#F4A259', '#7A5195', '#2D3047'],
    hoverlabel=dict(bgcolor='white', font_size=12),
    margin=dict(l=60, r=30, t=80, b=120),
)

```

```

def style_figure(fig, title, height=550):
    fig.update_layout(
        title=dict(text=title, x=0.02, xanchor='left'),
        height=height,
        legend=dict(orientation='h', y=-0.22, x=1, xanchor='right',
        ↪yanchor='top', font=dict(size=11)),
        hovermode='x unified',
        **BASE_LAYOUT,
    )
    return fig

def add_shaded_periods(fig, periods=SHADED_PERIODS):
    for start, end, label in periods:
        fig.add_vrect(
            x0=start, x1=end,
            fillcolor='rgba(125, 125, 125, 0.10)',
            line_width=0,
            annotation_text=label,
            annotation_position='top left',
            annotation_font=dict(size=8, color='rgba(100,100,100,0.55)'),
        )
    return fig

def axis_suffix(position):
    return '' if position == 1 else str(position)

```

## 1.4 2. Metrics and Data Preparation

Define the forecast evaluation metrics and the functions that compute realized variance from raw Alpaca bars, then clean and align IV, OVX, and TOSI onto the daily trading spine.

```

[ ]: # --- Forecast evaluation metrics ---

def rmse(y_true, y_pred):
    return float(np.sqrt(np.mean((np.asarray(y_true, float) - np.
    ↪asarray(y_pred, float)) ** 2)))

def directional_accuracy_from_change(actual_change, predicted_change):
    actual_change = np.asarray(actual_change, float)
    predicted_change = np.asarray(predicted_change, float)
    return float(np.mean(np.sign(actual_change) == np.sign(predicted_change)))

```

```

def sharpe_ratio(x, annualization=252):
    x = np.asarray(x, float)
    return float(x.mean() / (x.std(ddof=0) + 1e-10) * np.sqrt(annualization))

def evaluate_prediction_frame(pred_df):
    """
    Evaluate a fold-concatenated prediction frame whose target is the VRP.

    Expected columns in pred_df:
        vrp_actual          â€" actual log(RV_{t+1}) - log(IV_t), the VRP
    ↪target
        y_pred_vrp          â€" model's predicted VRP
        signal              â€" threshold-filtered directional signal (-1, 0,
    ↪+1)
        abs_daily_return_tplus1 â€" |close-to-close return on day t+1|, the
    ↪straddle payoff
        iv_daily_vol        â€" IV_t / 100 / sqrt(252), the daily implied vol
    ↪level
    """
    vrp_actual = pred_df['vrp_actual'].to_numpy()
    vrp_pred   = pred_df['y_pred_vrp'].to_numpy()
    signal     = pred_df['signal'].to_numpy()
    active     = signal != 0

    mae_val = float(np.mean(np.abs(vrp_actual - vrp_pred)))
    ss_res  = np.sum((vrp_actual - vrp_pred) ** 2)
    ss_tot  = np.sum((vrp_actual - vrp_actual.mean()) ** 2)

    # Economically accurate straddle P&L (unit notional, 1-day):
    #   Payoff = |close-to-close daily return_{t+1}| (includes overnight gap)
    #   Cost   = sqrt(2/Î€) Ã— iv_daily_vol (Black-Scholes ATM
    ↪straddle price)
    #   P&L    = signal Ã— (payoff âˆ’ cost)
    # Using actual |return| rather than sqrt(RV) gives true option economics:
    # the buyer receives the absolute move, not the intraday vol.
    if 'abs_daily_return_tplus1' in pred_df.columns and 'iv_daily_vol' in
    ↪pred_df.columns:
        abs_return = pred_df['abs_daily_return_tplus1'].to_numpy()
        straddle_cost = SQRT2_PI * pred_df['iv_daily_vol'].to_numpy()
        straddle_pnl = signal * (abs_return - straddle_cost)
        straddle_sharpe = sharpe_ratio(straddle_pnl)
        mean_straddle_pnl = float(straddle_pnl.mean())
        active_pnl = straddle_pnl[active]
    else:

```

```

    straddle_sharpe = np.nan
    mean_straddle_pnl = np.nan
    active_pnl = np.array([])

    return {
        'RMSE':          rmse(vrp_actual, vrp_pred),
        'MAE':           mae_val,
        'R2':            float(1.0 - ss_res / (ss_tot + 1e-10)),
        'Directional_Acc': float(np.mean(np.sign(vrp_pred) == np.
↪sign(vrp_actual))),
        # Base rate: fraction of days VRP is positive regardless of the model.
        # Directional_Acc is only meaningful when it materially exceeds
↪VRP_Pos_Base_Rate.
        'VRP_Pos_Base_Rate': float(np.mean(vrp_actual > 0)),
        'Pct_Days_Traded':  float(np.mean(active)),
        'Mean_Straddle_PnL': mean_straddle_pnl,
        'Sharpe_Straddle':  straddle_sharpe,
        'Signal_Hit_Rate':  float(np.mean(active_pnl > 0)) if active_pnl.size
↪else np.nan,
    }

def diebold_mariano(errors_a, errors_b, loss='mse'):
    """
    Two-sided Diebold-Mariano test for H0: equal forecast accuracy (h=1
↪horizon).
    Returns (DM statistic, p-value).
    Positive DM: model A has larger squared errors â†’ model B (extended) is
↪better.
    """
    e1, e2 = np.asarray(errors_a, float), np.asarray(errors_b, float)
    d = (e1 ** 2 - e2 ** 2) if loss == 'mse' else (np.abs(e1) - np.abs(e2))
    T = len(d)
    d_bar = d.mean()
    var_d = np.var(d, ddof=0) / T
    dm_stat = d_bar / np.sqrt(var_d + 1e-15)
    p_value = float(2 * (1 - stats.norm.cdf(abs(dm_stat))))
    return float(dm_stat), p_value

# --- Data loading and cleaning ---

def find_project_root(start=None):
    start = (start or Path.cwd()).resolve()
    for base in [start, *start.parents]:
        if (base / 'data').exists():
            return base

```

```

        nested = base / 'research' / 'Predicting-Airline-Stock-Volatility-main'
        if (nested / 'data').exists():
            return nested
        raise FileNotFoundError('Could not locate the project root containing the
↳data folder.')

def resolve_paths(start=None):
    project_root = find_project_root(start)
    data_dir = project_root / 'data'
    alpaca_dir = data_dir / 'alpaca_intraday'
    processed_dir = data_dir / 'processed'
    processed_dir.mkdir(parents=True, exist_ok=True)
    return project_root, data_dir, alpaca_dir, processed_dir

def compute_symbol_realized_variance(csv_path):
    intraday = pd.read_csv(
        csv_path,
        usecols=['symbol', 'trade_date', 'timestamp_et', 'close'],
        parse_dates=['timestamp_et'],
    )
    intraday['trade_date'] = pd.to_datetime(intraday['trade_date'])
    intraday = intraday.sort_values(['trade_date', 'timestamp_et']).copy()

    intraday['log_close'] = np.log(intraday['close'])
    intraday['intraday_log_return'] = intraday.
↳groupby('trade_date')['log_close'].diff()

    # Overnight gap:  $\log(\text{first\_bar\_close}_t) - \log(\text{last\_bar\_close}_{t-1})$ .
    # This captures price moves between sessions (earnings, macro events, etc.)
↳and
    # closes the bias between IV (24-hour market price) and intraday-only RV.
    day_bounds = (
        intraday.groupby(['symbol', 'trade_date'])
        .agg(log_day_open=('log_close', 'first'), log_day_close=('log_close',
↳'last'))
        .reset_index()
        .sort_values(['symbol', 'trade_date'])
    )
    day_bounds['log_prev_close'] = day_bounds.
↳groupby('symbol')['log_day_close'].shift(1)
    day_bounds['overnight_log_ret'] = day_bounds['log_day_open'] -
↳day_bounds['log_prev_close']
    day_bounds['overnight_rv'] = day_bounds['overnight_log_ret'] ** 2

```

```

    # close-to-close return used as the true straddle payoff ( $|R|$  is what the
    ↪ buyer receives)
    day_bounds['daily_log_return'] = day_bounds['log_day_close'] -
    ↪ day_bounds['log_prev_close']

    daily = (
        intraday.groupby(['symbol', 'trade_date'], as_index=False)
        .agg(
            rv_intraday=('intraday_log_return', lambda s: np.square(s.dropna()).
            ↪ sum()),
            n_intraday_returns=('intraday_log_return', lambda s: s.notna().
            ↪ sum()),
            n_bars=('close', 'size'),
        )
        .sort_values(['symbol', 'trade_date'])
        .reset_index(drop=True)
    )
    daily = daily.merge(
        day_bounds[['symbol', 'trade_date', 'overnight_rv',
        ↪ 'daily_log_return']],
        on=['symbol', 'trade_date'], how='left',
    )
    # Total daily RV = intraday sum-of-squares + overnight gap variance
    ↪ (Andersen et al. 2003)
    daily['rv_daily'] = daily['rv_intraday'] + daily['overnight_rv'].fillna(0)
    daily['abs_daily_return'] = daily['daily_log_return'].abs()

    daily = daily[daily['rv_intraday'] > 0].copy()
    daily['realized_vol_daily'] = np.sqrt(daily['rv_daily'])

    # Use mean-of-logs (Corsi 2009 log-HAR): rolling mean of log_rv_daily.
    daily['log_rv_daily'] = np.log(daily['rv_daily'])
    by_symbol_log = daily.groupby('symbol')['log_rv_daily']
    daily['log_rv_weekly'] = by_symbol_log.transform(
        lambda s: s.rolling(WEEK_WINDOW, min_periods=WEEK_WINDOW).mean()
    )
    daily['log_rv_monthly'] = by_symbol_log.transform(
        lambda s: s.rolling(MONTH_WINDOW, min_periods=MONTH_WINDOW).mean()
    )

    daily['target_log_rv_tplus1'] = daily.groupby('symbol')['log_rv_daily'].
    ↪ shift(-1)
    daily['target_change_tplus1'] = daily['target_log_rv_tplus1'] -
    ↪ daily['log_rv_daily']
    # Shift actual /return/ forward so row t carries the day-t+1 straddle payoff

```



```

    daily['abs_daily_return_tplus1'] = daily.
    ↪groupby('symbol')['abs_daily_return'].shift(-1)
    return daily

def build_realized_variance_panel_from_raw(alpaca_dir):
    csv_paths = [
        path
        for path in sorted(alpaca_dir.glob('*_5min_*.csv'))
        if not path.name.startswith('all_symbols') and path.stem.split('_')[0]_
    ↪in TICKERS
    ]
    if not csv_paths:
        raise FileNotFoundError('No raw Alpaca intraday files were found in_
    ↪data/alpaca_intraday/.')

    rv_panel = pd.concat((compute_symbol_realized_variance(path) for path in_
    ↪csv_paths), ignore_index=True)
    return rv_panel.sort_values(['symbol', 'trade_date']).reset_index(drop=True)

def load_iv_series(data_dir):
    iv_map = {}
    inventory_rows = []
    cleaning_rows = []

    for ticker in TICKERS:
        path = data_dir / f'{ticker}_IV.xlsx'
        raw = pd.read_excel(path, header=6)
        security_label = str(raw.columns[1]).strip()
        df = raw[['Date', 'HIST_CALL_IMP_VOL']]
    ↪rename(columns={'HIST_CALL_IMP_VOL': 'IV'})
        df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
        df['IV'] = pd.to_numeric(df['IV'], errors='coerce')
        rows_before = len(df)
        missing_before = int(df['IV'].isna().sum())
        df = df.sort_values('Date').dropna(subset=['Date', 'IV']).
    ↪reset_index(drop=True)
        iv_map[ticker] = df.set_index('Date')['IV']

    inventory_rows.append({
        'Dataset': f'{ticker} IV',
        'Rows': rows_before,
        'Columns': 1,
        'Start': df['Date'].min().date() if len(df) else np.nan,
        'End': df['Date'].max().date() if len(df) else np.nan,
    })

```

```

        'Security_Label': security_label,
        'Missing_Before_Cleaning': missing_before,
    })
    cleaning_rows.append({
        'Dataset': f'{ticker} IV',
        'Method': 'Skip metadata rows, parse dates/numbers, drop missing IV_
↪rows',
        'Rows_Removed': rows_before - len(df),
        'Missing_Before': missing_before,
        'Missing_After': int(df["IV"].isna().sum()),
    })

    return iv_map, pd.DataFrame(inventory_rows), pd.DataFrame(cleaning_rows)

def load_ovx_series(data_dir):
    # OVX_Price.xlsx follows the same Bloomberg layout as the IV workbooks:
    # 6 metadata rows, then Date / PX_LAST / CHG_PCT_1D.
    path = data_dir / 'OVX_Price.xlsx'
    raw = pd.read_excel(path, header=6)
    df = raw[['Date', 'PX_LAST']].rename(columns={'PX_LAST': 'OVX'})
    df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
    df['OVX'] = pd.to_numeric(df['OVX'], errors='coerce')
    rows_before = len(df)
    missing_before = int(df['OVX'].isna().sum())
    ovx = df.dropna().sort_values('Date').reset_index(drop=True)

    inventory = pd.DataFrame([
        'Dataset': 'OVX', 'Rows': rows_before, 'Columns': 1,
        'Start': ovx['Date'].min().date(), 'End': ovx['Date'].max().date(),
        'Missing_Before_Cleaning': missing_before,
    ])
    cleaning = pd.DataFrame([
        'Dataset': 'OVX', 'Method': 'Read OVX_Price.xlsx (Bloomberg layout),
↪drop missing OVX values',
        'Rows_Removed': rows_before - len(ovx),
        'Missing_Before': missing_before, 'Missing_After': int(ovx['OVX'].
↪isna().sum()),
    ])
    return ovx.set_index('Date')['OVX'], inventory, cleaning

def load_tosi_series(data_dir):
    tosi = pd.read_csv(data_dir / 'TOSI.csv', usecols=['Date', 'TOSI'])
    tosi['Date'] = pd.to_datetime(tosi['Date'], format='%b-%y', errors='coerce')
    tosi['TOSI'] = pd.to_numeric(tosi['TOSI'], errors='coerce')
    missing_before = int(tosi['TOSI'].isna().sum())

```

```

tosi = tosi.dropna().sort_values('Date').reset_index(drop=True)

tosi['Date'] = tosi['Date'] + pd.offsets.MonthBegin(1)
tosi_level = tosi.set_index('Date')['TOSI']
tosi_change = tosi_level.diff()

inventory = pd.DataFrame([
    'Dataset': 'TOSI', 'Rows': len(tosi), 'Columns': 1,
    'Start': tosi['Date'].min().date(), 'End': tosi['Date'].max().date(),
    'Missing_Before_Cleaning': missing_before,
])

cleaning = pd.DataFrame([
    'Dataset': 'TOSI', 'Method': 'Shift forward one month, then_
↳forward-fill to daily spine',
    'Rows_Removed': 0,
    'Missing_Before': missing_before, 'Missing_After': int(tosi['TOSI'].
↳isna().sum()),
])

return tosi_level, tosi_change, inventory, cleaning

def build_model_panel(ticker, rv_panel, iv_map, ovx, tosi_level, tosi_change):
    panel = (
        rv_panel.loc[rv_panel['symbol'] == ticker]
        .copy()
        .sort_values('trade_date')
        .reset_index(drop=True)
    )

    spine = pd.Index(panel['trade_date'])
    panel['IV'] = iv_map[ticker].reindex(spine).ffill(limit=3).to_numpy()
    panel['OVX'] = ovx.reindex(spine).ffill(limit=3).to_numpy()
    panel['tosi_level'] = tosi_level.reindex(spine, method='ffill').to_numpy()
    panel['tosi_change'] = tosi_change.reindex(spine, method='ffill').to_numpy()

    # IV is annualised percentage vol (e.g. 30 = 30% p.a.).
    # Convert to daily variance before logging so it is on the same scale as_
↳log_rv_daily.
    iv_daily_var = (panel['IV'] / 100) ** 2 / 252
    panel['iv_daily_vol'] = panel['IV'] / 100 / np.sqrt(252)
    panel['log_iv_daily'] = np.log(iv_daily_var.clip(lower=1e-10))

    # OVX is also an annualised percentage vol index â€" same conversion.
    ovx_daily_var = (panel['OVX'] / 100) ** 2 / 252
    panel['log_ovx_daily'] = np.log(ovx_daily_var.clip(lower=1e-10))
    panel['log_ovx_weekly'] = panel['log_ovx_daily'].rolling(WEEK_WINDOW,
↳min_periods=WEEK_WINDOW).mean()

```

```

    panel['log_ovx_monthly'] = panel['log_ovx_daily'].rolling(MONTH_WINDOW,
↳min_periods=MONTH_WINDOW).mean()

    # VRP target:  $\log(RV_{t+1}) - \log(IV_t)$ , both on daily-variance scale.
    # A positive VRP means realized vol exceeded implied vol â€œ the straddle
↳buyer profits.
    panel['vrp_target'] = panel['target_log_rv_tplus1'] - panel['log_iv_daily']

    return panel

```

```

def build_all_panels(rv_panel, iv_map, ovx, tosi_level, tosi_change):
    return {
        ticker: build_model_panel(ticker, rv_panel, iv_map, ovx, tosi_level,
↳tosi_change)
        for ticker in TICKERS
    }

```

```

def build_monthly_exploration_frame(panels):
    rows = []
    for ticker, panel in panels.items():
        monthly = (
            panel.set_index('trade_date')
                .resample('MS')
                .agg({
                    'log_rv_daily': 'mean',
                    'log_iv_daily': 'mean',
                    'log_ovx_daily': 'mean',
                    'tosi_level': 'last',
                })
            .rename(columns={
                'log_rv_daily': f'{ticker}_log_rv',
                'log_iv_daily': f'{ticker}_log_iv',
            })
        )
        rows.append(monthly)
    merged = pd.concat(rows, axis=1)
    merged = merged.loc[:, ~merged.columns.duplicated()].sort_index()
    return merged

```

```

[ ]: project_root, data_dir, alpaca_dir, processed_dir = resolve_paths()
rv_panel = build_realized_variance_panel_from_raw(alpaca_dir)
iv_map, iv_inventory, iv_cleaning = load_iv_series(data_dir)
ovx, ovx_inventory, ovx_cleaning = load_ovx_series(data_dir)
tosi_level, tosi_change, tosi_inventory, tosi_cleaning =
↳load_tosi_series(data_dir)

```

```

panels = build_all_panels(rv_panel, iv_map, ovx, tosi_level, tosi_change)
monthly_exploration = build_monthly_exploration_frame(panels)

rv_inventory = (
    rv_panel.groupby('symbol', as_index=False)
    .agg(
        Rows=('trade_date', 'size'),
        Start=('trade_date', 'min'),
        End=('trade_date', 'max'),
        Min_Realized_Vol=('realized_vol_daily', 'min'),
        Max_Realized_Vol=('realized_vol_daily', 'max'),
        Min_Intraday_Returns=('n_intraday_returns', 'min'),
        Max_Intraday_Returns=('n_intraday_returns', 'max'),
    )
    .rename(columns={'symbol': 'Dataset'})
)
rv_inventory['Dataset'] = rv_inventory['Dataset'].astype(str) + ' RV'
rv_inventory['Columns'] = 8
rv_inventory['Missing_Before_Cleaning'] = 0

rv_cleaning = pd.DataFrame([
    'Dataset': 'Alpaca intraday bars',
    'Method': 'Compute intraday log returns, sum squared returns within each_
↳ day, build rolling 5/22-day HAR features',
    'Rows_Removed': 0,
    'Missing_Before': 0,
    'Missing_After': 0,
])

data_inventory = pd.concat([
    rv_inventory[['Dataset', 'Rows', 'Columns', 'Start', 'End',_
↳ 'Missing_Before_Cleaning']],
    iv_inventory[['Dataset', 'Rows', 'Columns', 'Start', 'End',_
↳ 'Missing_Before_Cleaning']],
    ovx_inventory[['Dataset', 'Rows', 'Columns', 'Start', 'End',_
↳ 'Missing_Before_Cleaning']],
    tosi_inventory[['Dataset', 'Rows', 'Columns', 'Start', 'End',_
↳ 'Missing_Before_Cleaning']],
], ignore_index=True)

cleaning_log = pd.concat([rv_cleaning, iv_cleaning, ovx_cleaning,_
↳ tosi_cleaning], ignore_index=True)

feature_map = pd.DataFrame({
    'Feature_Spec': list(FEATURE_SPECS.keys()),
    'Features': [' ', '.join(cols) for cols in FEATURE_SPECS.values()],

```

```

        'Feature_Count': [len(cols) for cols in FEATURE_SPECS.values()],
    })

print(f'Project root: {project_root}')
print('Data inventory')
display(data_inventory)

print('Cleaning log')
display(cleaning_log)

print('Feature map')
display(feature_map)

```

### 1.5 3. Modeling Utilities

Create the expanding-window walk-forward helpers and the model factories for OLS, Ridge, Random Forest, and XGBoost.

```

[ ]: # --- Walk-forward utilities ---

def make_linear_model(family_name):
    if family_name == 'OLS':
        return LinearRegression()
    if family_name == 'Ridge':
        return Pipeline([('scaler', StandardScaler()), ('model', Ridge(alpha=1.
↪0))])
    raise ValueError(f'Not a linear family: {family_name}')

def tune_and_fit_tree(family_name, X_train, y_train):
    """
    Nested CV for tree models: hold out the most recent val_size rows of the
    training window, grid-search over key hyperparameters, refit on the full
    training window with the best params.
    """
    val_size = max(WF_TEST_DAYS, len(X_train) // 5)
    X_tr, y_tr = X_train[:-val_size], y_train[:-val_size]
    X_val, y_val = X_train[-val_size:], y_train[-val_size:]

    best_rmse, best_params = np.inf, {}

    if family_name == 'Random Forest':
        grid = [
            {'max_depth': d, 'min_samples_leaf': 1}
            for d in [4, 6, 8]
            for l in [3, 5, 8]
        ]

```

```

        for params in grid:
            m = RandomForestRegressor(
                n_estimators=200, max_features=0.8, n_jobs=-1, random_state=42,
↪**params
            )
            m.fit(X_tr, y_tr)
            val_rmse = np.sqrt(np.mean((y_val - m.predict(X_val)) ** 2))
            if val_rmse < best_rmse:
                best_rmse, best_params = val_rmse, params
            model = RandomForestRegressor(
                n_estimators=300, max_features=0.8, n_jobs=-1, random_state=42,
↪**best_params
            )

    elif family_name == 'XGBoost':
        grid = [
            {'max_depth': d, 'learning_rate': lr}
            for d in [2, 3, 4]
            for lr in [0.03, 0.07]
        ]
        for params in grid:
            m = XGBRegressor(
                n_estimators=200, subsample=0.8, colsample_bytree=0.8,
                reg_lambda=1.0, min_child_weight=5, verbosity=0,
                random_state=42, n_jobs=4, objective='reg:squarederror',
↪**params
            )
            m.fit(X_tr, y_tr)
            val_rmse = np.sqrt(np.mean((y_val - m.predict(X_val)) ** 2))
            if val_rmse < best_rmse:
                best_rmse, best_params = val_rmse, params
            model = XGBRegressor(
                n_estimators=300, subsample=0.8, colsample_bytree=0.8,
                reg_lambda=1.0, min_child_weight=5, verbosity=0,
                random_state=42, n_jobs=4, objective='reg:squarederror',
↪**best_params
            )
        else:
            raise ValueError(f'Not a tree family: {family_name}')

    model.fit(X_train, y_train)
    return model

def walk_forward_splits(n_obs, min_train=WF_MIN_TRAIN_DAYS,
↪test_size=WF_TEST_DAYS, step_size=WF_STEP_DAYS):
    start = min_train

```

```

while start + test_size <= n_obs:
    yield slice(0, start), slice(start, start + test_size)
    start += step_size

def _select_threshold(pred_vrp_grid, pred_vrp_val, val_abs_return, val_iv_vol):
    """
    Search over a grid of candidate thresholds and return the one that maximises
    straddle Sharpe on the held-out validation sub-window predictions.

    Parameters
    -----
    pred_vrp_grid : predictions on the pre-validation training portion â€”
                  used only to build the percentile grid, never evaluated.
    pred_vrp_val   : OOS predictions on the validation sub-window (truly
    ↪out-of-sample
                  because the model was fit only on the pre-validation data).
    val_abs_return : actual |close-to-close return_{t+1}| for the validation
    ↪period
    val_iv_vol      : iv_daily_vol for the validation period (ATM straddle cost
    ↪basis)

    Only thresholds producing at least MIN_TRADES_FOR_SHARPE trades are
    ↪eligible.
    """
    abs_pred_grid = np.abs(pred_vrp_grid)
    candidates = np.unique([np.percentile(abs_pred_grid, p) for p in
    ↪THRESHOLD_PERCENTILES])

    best_sharpe = -np.inf
    best_thresh = candidates[0]
    best_pct_traded = np.nan

    for thresh in candidates:
        sig = np.where(np.abs(pred_vrp_val) > thresh, np.sign(pred_vrp_val), 0.
        ↪0)

        n_traded = int(np.sum(sig != 0))
        if n_traded < MIN_TRADES_FOR_SHARPE:
            continue
        pnl = sig * (val_abs_return - SQRT2_PI * val_iv_vol)
        sr = sharpe_ratio(pnl)
        if sr > best_sharpe:
            best_sharpe = sr
            best_thresh = thresh
            best_pct_traded = float(n_traded / len(sig))

```



```

return best_thresh, best_sharpe, best_pct_traded

def run_model_for_spec_family(panel, feature_cols, family_name):
    """
    Expanding-window walk-forward loop. Target is  $VRP = \log(RV_{t+1}) - \log(IV_t)$ .

    Threshold selection is fully nested:
    1. Split each training window into pre-val (80%) and val (20%) portions.
    2. Fit model ONLY on pre-val data and predict on val (truly OOS within training).
    3. Select the threshold that maximises straddle Sharpe on those OOS val predictions.
    4. Refit model on the full training window for the actual held-out test predictions.

    This eliminates the in-sample contamination that arises when the threshold is tuned
    on predictions made by a model that already saw the validation rows during training.

    Returns
    -----
    pred_df : pd.DataFrame
        Fold-concatenated predictions with signal and threshold columns.
    metrics : dict
        Aggregate OOS metrics.
    threshold_log_df : pd.DataFrame
        Per-fold threshold, validation Sharpe, and fraction of days traded.
    """
    always_needed = [
        'vrp_target', 'iv_daily_vol', 'log_iv_daily', 'log_rv_daily',
        'target_log_rv_tplus1', 'target_change_tplus1',
        'abs_daily_return_tplus1',
    ]
    needed_strict = [c for c in feature_cols + always_needed if c in panel.columns]
    work = panel.dropna(subset=needed_strict).copy().reset_index(drop=True)

    if len(work) < WF_MIN_TRAIN_DAYS + WF_TEST_DAYS:
        return None, None, None

    is_tree = family_name in ('Random Forest', 'XGBoost')
    fold_predictions = []
    fold_threshold_log = []
    n_folds = 0

```

```

for train_slice, test_slice in walk_forward_splits(len(work)):
    train = work.iloc[train_slice]
    test = work.iloc[test_slice].copy()

    X_tr = train[feature_cols].to_numpy()
    y_tr = train['vrp_target'].to_numpy()

    # Nested threshold selection
    ↪ # Split training into pre-val and val so the model never sees val rows
    # during fitting val predictions are genuinely out-of-sample.
    val_size = max(WF_TEST_DAYS, len(X_tr) // 5)
    X_tr_only, y_tr_only = X_tr[:-val_size], y_tr[:-val_size]
    X_val = X_tr[-val_size:]
    val_data = train.iloc[-val_size:]

    if is_tree:
        model_for_val = tune_and_fit_tree(family_name, X_tr_only, y_tr_only)
    else:
        model_for_val = make_linear_model(family_name)
        model_for_val.fit(X_tr_only, y_tr_only)

    pred_vrp_grid = model_for_val.predict(X_tr_only) # percentile grid
    ↪ basis
    pred_vrp_val = model_for_val.predict(X_val) # truly OOS val
    ↪ predictions

    best_thresh, best_sharpe_val, pct_traded_val = _select_threshold(
        pred_vrp_grid,
        pred_vrp_val,
        val_data['abs_daily_return_tplus1'].to_numpy(),
        val_data['iv_daily_vol'].to_numpy(),
    )

    # Refit on full training window for OOS test predictions
    ↪ #
    if is_tree:
        model = tune_and_fit_tree(family_name, X_tr, y_tr)
    else:
        model = make_linear_model(family_name)
        model.fit(X_tr, y_tr)

    pred_vrp_test = model.predict(test[feature_cols].to_numpy())
    test = test.copy()
    test['y_pred_vrp'] = pred_vrp_test
    test['vrp_actual'] = test['vrp_target']

```

```

test['threshold'] = best_thresh
test['signal'] = np.where(
    np.abs(pred_vrp_test) > best_thresh, np.sign(pred_vrp_test), 0.0
)
test['fold_id'] = n_folds + 1

pct_traded_test = float((test['signal'] != 0).mean())

fold_threshold_log.append({
    'fold_id': n_folds + 1,
    'threshold': best_thresh,
    'pct_traded_val': pct_traded_val,
    'pct_traded_test': pct_traded_test,
    'val_sharpe': best_sharpe_val,
})

save_cols = [
    'trade_date', 'symbol', 'log_rv_daily',
    'target_log_rv_tplus1', 'target_change_tplus1',
    'abs_daily_return_tplus1',
    'vrp_actual', 'y_pred_vrp', 'signal', 'threshold', 'fold_id',
]

if 'iv_daily_vol' in test.columns:
    save_cols.append('iv_daily_vol')

fold_predictions.append(test[save_cols])
n_folds += 1

pred_df = pd.concat(fold_predictions, ignore_index=True)
threshold_log_df = pd.DataFrame(fold_threshold_log)
metrics = evaluate_prediction_frame(pred_df)
metrics['N_Predictions'] = int(len(pred_df))
metrics['N_Folds'] = int(n_folds)
metrics['OOS_Start'] = pred_df['trade_date'].min()
metrics['OOS_End'] = pred_df['trade_date'].max()
metrics['Avg_Threshold'] = float(threshold_log_df['threshold'].mean())
metrics['Avg_Pct_Traded'] = float(threshold_log_df['pct_traded_test'].
    ↪mean())
return pred_df, metrics, threshold_log_df

def compute_full_sample_feature_importance(panel, feature_cols, family_name):
    if family_name not in ['Random Forest', 'XGBoost']:
        return None

    work = panel.dropna(subset=feature_cols + ['vrp_target']).copy()
    if work.empty:

```

```

        return None

    X = work[feature_cols].to_numpy()
    y = work['vrp_target'].to_numpy()
    model = tune_and_fit_tree(family_name, X, y)
    return pd.Series(model.feature_importances_, index=feature_cols).
    ↪sort_values(ascending=False)

```

## 1.6 4. Train and Evaluate Models

Fit each model family under four feature specifications: **HAR-RV**, **HAR-RV + IV**, **HAR-RV + IV + OVX**, and **HAR-RV + IV + OVX + TOSI**. All models are scored on the same expanding walk-forward out-of-sample windows.

```

[ ]: results_rows = []
prediction_frames = []
all_threshold_logs = []    # collect per-fold threshold metadata across all runs
feature_importances = {}

for ticker, panel in panels.items():
    baseline_rmse = np.nan
    baseline_sharpe = np.nan

    for feature_spec, feature_cols in FEATURE_SPECS.items():
        for family_name in MODEL_FAMILIES:
            pred_df, metrics, threshold_log_df = run_model_for_spec_family(
                panel, feature_cols, family_name
            )
            if pred_df is None or metrics is None:
                continue

            pred_df = pred_df.copy()
            pred_df['Ticker'] = ticker
            pred_df['Feature_Spec'] = feature_spec
            pred_df['Model_Family'] = family_name
            prediction_frames.append(pred_df)

            # Annotate threshold log with run identifiers.
            tlog = threshold_log_df.copy()
            tlog['Ticker'] = ticker
            tlog['Feature_Spec'] = feature_spec
            tlog['Model_Family'] = family_name
            all_threshold_logs.append(tlog)

            # Warn if average fraction of days traded is below the floor.
            avg_traded = metrics['Avg_Pct_Traded']
            if avg_traded < TRADE_FRAC_WARN:

```

```

        print(
            f' WARNING low trade rate: {ticker} | {feature_spec} |␣
↪{family_name} '
            f'avg_pct_traded={avg_traded:.1%} (< {TRADE_FRAC_WARN:.0%}).
↪ '
            'Sharpe estimate may be unreliable.'
        )

    row = {
        'Ticker': ticker,
        'Feature_Spec': feature_spec,
        'Model_Family': family_name,
        'Model': f'{family_name} | {feature_spec}',
        **metrics,
    }

    if feature_spec == 'HAR-RV' and family_name == 'OLS':
        baseline_rmse = metrics['RMSE']
        baseline_sharpe = metrics['Sharpe_Straddle']

    row['RMSE_vs_OLS_HAR_RV'] = (
        metrics['RMSE'] / baseline_rmse if not np.isnan(baseline_rmse)␣
↪else np.nan
    )
    row['Sharpe_vs_OLS_HAR_RV'] = (
        metrics['Sharpe_Straddle'] - baseline_sharpe
        if not np.isnan(baseline_sharpe) else np.nan
    )
    results_rows.append(row)

    if feature_spec == 'HAR-RV+IV+OVX+TOSI':
        feature_importances.setdefault(ticker, {})
        feature_importances[ticker]['Random Forest'] =␣
↪compute_full_sample_feature_importance(
            panel, feature_cols, 'Random Forest'
        )
        feature_importances[ticker]['XGBoost'] =␣
↪compute_full_sample_feature_importance(
            panel, feature_cols, 'XGBoost'
        )

results_df = pd.DataFrame(results_rows)
predictions_df = pd.concat(prediction_frames, ignore_index=True)
threshold_log_all_df = pd.concat(all_threshold_logs, ignore_index=True)

summary_df = (
    results_df.groupby(['Feature_Spec', 'Model_Family'], as_index=False)

```

```

    .agg(
        Avg_RMSE           = ('RMSE',           'mean'),
        Avg_MAE            = ('MAE',           'mean'),
        Avg_R2             = ('R2',            'mean'),
        Avg_Directional_Acc = ('Directional_Acc', 'mean'),
        Avg_Pct_Days_Traded = ('Pct_Days_Traded', 'mean'),
        Avg_Sharpe_Straddle = ('Sharpe_Straddle', 'mean'),
        Avg_RMSE_vs_OLS_HAR_RV = ('RMSE_vs_OLS_HAR_RV', 'mean'),
        Avg_Sharpe_vs_OLS_HAR_RV = ('Sharpe_vs_OLS_HAR_RV', 'mean'),
    )
    .sort_values(['Feature_Spec', 'Avg_RMSE', 'Avg_Sharpe_Straddle'],
        ascending=[True, True, False])
    .reset_index(drop=True)
)

family_summary_df = (
    results_df.groupby('Model_Family', as_index=False)
    .agg(
        Avg_RMSE           = ('RMSE',           'mean'),
        Avg_R2             = ('R2',            'mean'),
        Avg_Directional_Acc = ('Directional_Acc', 'mean'),
        Avg_Pct_Days_Traded = ('Pct_Days_Traded', 'mean'),
        Avg_Sharpe_Straddle = ('Sharpe_Straddle', 'mean'),
    )
    .sort_values(['Avg_RMSE', 'Avg_Sharpe_Straddle'], ascending=[True, False])
    .reset_index(drop=True)
)

ridge_vs_ols_df = (
    results_df[results_df['Model_Family'].isin(['OLS', 'Ridge'])]
    .groupby(['Feature_Spec', 'Model_Family'], as_index=False)
    .agg(
        Avg_RMSE           = ('RMSE',           'mean'),
        Avg_R2             = ('R2',            'mean'),
        Avg_Directional_Acc = ('Directional_Acc', 'mean'),
        Avg_Pct_Days_Traded = ('Pct_Days_Traded', 'mean'),
        Avg_Sharpe_Straddle = ('Sharpe_Straddle', 'mean'),
    )
)

best_models_df = (
    results_df.loc[results_df.groupby('Ticker')['RMSE'].idxmin()]
    .sort_values('Ticker')
    .reset_index(drop=True)
)

```

```

# Per-fold threshold summary: how did the chosen threshold and trade-rate
↪ evolve?
threshold_summary_df = (
    threshold_log_all_df
    .groupby(['Ticker', 'Feature_Spec', 'Model_Family'], as_index=False)
    .agg(
        N_Folds          = ('fold_id',          'count'),
        Avg_Threshold     = ('threshold',        'mean'),
        Std_Threshold     = ('threshold',        'std'),
        Avg_Pct_Traded_Val = ('pct_traded_val',  'mean'),
        Avg_Pct_Traded_Test = ('pct_traded_test', 'mean'),
        Avg_Val_Sharpe    = ('val_sharpe',      'mean'),
        Min_Pct_Traded    = ('pct_traded_test',  'min'),
        Max_Pct_Traded    = ('pct_traded_test',  'max'),
    )
)
threshold_summary_df['Low_Trade_Rate_Flag'] = (
    threshold_summary_df['Avg_Pct_Traded_Test'] < TRADE_FRAC_WARN
)

# --- Diebold-Mariano tests (errors on VRP, OLS models only) ---
# Test incremental value of each additional variable along both the no-IV and
↪ IV branches,
# plus a cross-branch test of IV vs no-IV for the OVX specs.
spec_pairs = [
    # No-IV branch
    ('HAR-RV',          'HAR-RV+OVX',          'OVX adds to HAR-RV?'),
    ('HAR-RV+OVX',      'HAR-RV+OVX+TOSI',      'TOSI adds to HAR-RV+OVX?'),
    # IV branch
    ('HAR-RV',          'HAR-RV+IV',           'IV adds to HAR-RV?'),
    ('HAR-RV+IV',       'HAR-RV+IV+OVX',       'OVX adds to HAR-RV+IV?'),
    ('HAR-RV+IV+OVX',   'HAR-RV+IV+OVX+TOSI',   'TOSI adds to HAR-RV+IV+OVX?'),
    # Cross-branch: does IV matter once OVX is in?
    ('HAR-RV+OVX',      'HAR-RV+IV+OVX',       'IV adds to HAR-RV+OVX?'),
]

dm_rows = []
for ticker in TICKERS:
    ticker_preds = predictions_df[
        (predictions_df['Ticker'] == ticker) &
        (predictions_df['Model_Family'] == 'OLS')
    ]
    for spec_a, spec_b, hypothesis in spec_pairs:
        preds_a = ticker_preds[ticker_preds['Feature_Spec'] == spec_a].
        ↪ set_index('trade_date')
        preds_b = ticker_preds[ticker_preds['Feature_Spec'] == spec_b].
        ↪ set_index('trade_date')

```

```

common = preds_a.index.intersection(preds_b.index)
if len(common) < 30:
    continue
e_a = preds_a.loc[common, 'vrp_actual'] - preds_a.loc[common,
↪'y_pred_vrp']
e_b = preds_b.loc[common, 'vrp_actual'] - preds_b.loc[common,
↪'y_pred_vrp']
dm_stat, p_val = diebold_mariano(e_a.to_numpy(), e_b.to_numpy())
dm_rows.append({
    'Ticker':      ticker,
    'Baseline':    spec_a,
    'Extended':    spec_b,
    'Hypothesis':  hypothesis,
    'DM_Stat':     dm_stat,
    'P_Value':     p_val,
    'Significant_5pct': p_val < 0.05,
    'Better_Model': spec_b if dm_stat > 0 else spec_a,
})

dm_results_df = pd.DataFrame(dm_rows)

visual_catalog = pd.DataFrame([
    {'Visual': '1. Daily realized volatility lines',          ↵
↪'Interactive_Element': 'Range slider + shaded periods + unified hover',↵
↪'Annotated': 'Yes'},
    {'Visual': '2. Oil driver figure',                        ↵
↪'Interactive_Element': 'Dual-axis hover + monthly markers + annotation',↵
↪'Annotated': 'Yes'},
    {'Visual': '3. Monthly correlation heatmap',              ↵
↪'Interactive_Element': 'Hover lookup + text labels',          ↵
↪'Annotated': 'Yes'},
    {'Visual': '4. IV vs next-day log RV scatter facets',    ↵
↪'Interactive_Element': 'Facet hover + fitted lines',          ↵
↪'Annotated': 'Yes'},
    {'Visual': '5. OVX vs next-month log RV scatter↵
↪facets', 'Interactive_Element': 'Facet hover + cubic fits',    ↵
↪'Annotated': 'Yes'},
    {'Visual': '6. TOSI vs next-month log RV scatter↵
↪facets', 'Interactive_Element': 'Facet hover + linear fits',    ↵
↪'Annotated': 'Yes'},
    {'Visual': '7. Per-ticker RMSE comparison (VRP↵
↪target)', 'Interactive_Element': '2Ã2 ticker panels + faded non-best +↵
↪hover', 'Annotated': 'Yes'},
    {'Visual': '8. Average straddle Sharpe comparison',      ↵
↪'Interactive_Element': 'Grouped bars + best-model annotation',  ↵
↪'Annotated': 'Yes'},

```



```

        {'Visual': '9. Forecast comparison panels',
         ↪'Interactive_Element': 'Best-spec per ticker + RMSE annotation',
         ↪'Annotated': 'Yes'},
        {'Visual': '10. Ridge vs OLS comparison',
         ↪'Interactive_Element': 'Two-metric side-by-side bars',
         ↪'Annotated': 'Yes'},
        {'Visual': '11. Tree-based feature importance (VRP)',
         ↪'Interactive_Element': 'Horizontal bars + hover',
         ↪'Annotated': 'Yes'},
    ])

results_snapshot_df = results_df[[
    'Ticker', 'Feature_Spec', 'Model_Family', 'Model', 'RMSE', 'MAE', 'R2',
    'Directional_Acc', 'Pct_Days_Traded', 'Avg_Threshold', 'Avg_Pct_Traded',
    'Sharpe_Straddle', 'Mean_Straddle_PnL', 'Signal_Hit_Rate',
    'RMSE_vs_OLS_HAR_RV', 'Sharpe_vs_OLS_HAR_RV', 'N_Folds', 'N_Predictions'
]].sort_values(['Ticker', 'RMSE', 'Sharpe_Straddle'], ascending=[True, True,
↪False]).reset_index(drop=True)

```

## 1.7 5. Interactive Visual Design

Build interactive Plotly figures in the same storytelling style as the reference notebook: exploratory views first, then model-comparison visuals, each with consistent airline colors, model colors, annotations, and hover detail.

```

[ ]: # --- Shared chart utilities ---

SPEC_SHORT = {
    'HAR-RV': 'HAR-RV',
    'HAR-RV+OVX': '+OVX',
    'HAR-RV+OVX+TOSI': '+OVX+TOSI',
    'HAR-RV+IV': '+IV',
    'HAR-RV+IV+OVX': '+IV+OVX',
    'HAR-RV+IV+OVX+TOSI': '+IV+OVX+TOSI',
}

def polynomial_r2(x, y, degree):
    coeffs = np.polyfit(x, y, degree)
    poly = np.poly1d(coeffs)
    y_hat = poly(x)
    ss_res = np.sum((y - y_hat) ** 2)
    ss_tot = np.sum((y - y.mean()) ** 2)
    return coeffs, poly, float(1.0 - ss_res / (ss_tot + 1e-10))

def build_monthly_driver_pair(panel, driver_col, agg='mean'):
    monthly = (

```

```

        panel.set_index('trade_date')
        .resample('MS')
        .agg({driver_col: agg, 'log_rv_daily': 'mean'})
        .rename(columns={'log_rv_daily': 'monthly_log_rv'})
    )
    monthly['target_next_month_log_rv'] = monthly['monthly_log_rv'].shift(-1)
    return monthly.dropna(subset=[driver_col, 'target_next_month_log_rv']).
↪copy()

# --- Exploratory figures ---

def make_sector_realized_vol_figure():
    fig = go.Figure()
    for ticker in TICKERS:
        df = panels[ticker].copy()
        df['rv_21d'] = df['realized_vol_daily'].rolling(21, min_periods=10).
↪mean()
        fig.add_trace(go.Scatter(
            x=df['trade_date'], y=df['rv_21d'],
            mode='lines', name=ticker,
            line=dict(color=AIRLINE_COLORS[ticker], width=2),
            hovertemplate=f'{ticker}<br>{{x|%Y-%m-%d}}<br>21D realized_
↪vol={{y:.3f}}<extra></extra>',
            ))
        add_shaded_periods(fig)
        fig.update_xaxes(rangeslider_visible=True, title='Date')
        fig.update_yaxes(title='21-day average realized volatility')
        fig.add_annotation(
            x='2022-06-15', y=0.080,
            text='Airline realized volatility rose sharply during the 2022 energy_
↪shock',
            showarrow=True, arrowhead=2, bgcolor='rgba(255,255,255,0.85)',
        )
        return style_figure(fig, 'Daily Realized Volatility Across Airlines and_
↪JETS', height=620)

def make_oil_driver_figure():
    sector_rv = rv_panel.groupby('trade_date')['realized_vol_daily'].mean().
↪rename('Sector_RV').reset_index()
    tosi_df = tosi_level.reset_index(name='TOSI'); tosi_df.columns = ['Date',_
↪'TOSI']
    ovx_df = ovx.reset_index(name='OVX'); ovx_df.columns = ['Date', 'OVX']

    fig = make_subplots(specs=[[{'secondary_y': True}]]))

```

```

fig.add_trace(go.Scatter(x=ovx_df['Date'], y=ovx_df['OVX'], mode='lines',
↪name='OVX',
    line=dict(color='#C75146', width=2),
    hovertemplate='OVX<br>{x|%Y-%m-%d}<br>{y:.2f}<extra></extra>'),
↪secondary_y=False)
fig.add_trace(go.Scatter(x=sector_rv['trade_date'],
↪y=sector_rv['Sector_RV'], mode='lines', name='Sector RV',
    line=dict(color='#2D3047', width=2),
    hovertemplate='Sector RV<br>{x|%Y-%m-%d}<br>{y:.3f}<extra></extra>'),
↪secondary_y=False)
fig.add_trace(go.Scatter(x=tosi_df['Date'], y=tosi_df['TOSI'],
↪mode='lines+markers', name='TOSI',
    line=dict(color='#3C91E6', width=2.5), marker=dict(size=5),
    hovertemplate='TOSI<br>{x|%Y-%m-%d}<br>{y:.2f}<extra></extra>'),
↪secondary_y=True)

add_shaded_periods(fig, periods=[('2022-02-01', '2022-08-31', 'Energy
↪shock')]))
fig.add_hline(y=0, line_dash='dash', line_color='#7F8C8D', secondary_y=True)
fig.update_xaxes(title='Date')
fig.update_yaxes(title='OVX / sector realized volatility',
↪secondary_y=False)
fig.update_yaxes(title='TOSI sentiment index', secondary_y=True)
fig.add_annotation(x=0.99, y=0.99, xref='paper', yref='paper',
↪xanchor='right', yanchor='top',
    text='OVX and average airline realized volatility moved together most
↪strongly during the 2022 oil shock',
    showarrow=False, font=dict(size=11), bgcolor='rgba(255,255,255,0.85)')
return style_figure(fig, 'Oil Volatility, Oil Sentiment, and Sector
↪Realized Volatility', height=600)

def make_correlation_heatmap():
    corr_frame = monthly_exploration[[
        'DAL_log_rv', 'DAL_log_iv', 'UAL_log_rv', 'UAL_log_iv',
        'LUV_log_rv', 'LUV_log_iv', 'JETS_log_rv', 'JETS_log_iv',
        'log_ovx_daily', 'tosi_level'
    ]].dropna()
    corr = corr_frame.corr()
    labels = ['DAL RV', 'DAL IV', 'UAL RV', 'UAL IV', 'LUV RV', 'LUV IV', 'JETS
↪RV', 'JETS IV', 'OVX', 'TOSI']

    fig = go.Figure(go.Heatmap(
        z=corr.values, x=labels, y=labels, zmid=0, zmin=-1, zmax=1,
        colorscale='RdBu', text=np.round(corr.values, 2),
↪texttemplate='%{text}',

```

```

        hovertemplate='%{x} vs %{y}<br>Pearson r=%{z:.2f}<extra></extra>',
        colorbar=dict(title='Pearson r'),
    ))
    fig2 = style_figure(fig, 'Monthly Correlation Structure for Realized_
↳Volatility, IV, OVX, and TOSI', height=650)
    fig2.update_layout(margin=dict(l=60, r=30, t=110, b=120))
    return fig2

def make_iv_scatter_figure():
    positions = [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2)]
    fig = make_subplots(rows=2, cols=3, subplot_titles=TICKERS,
↳vertical_spacing=0.14, horizontal_spacing=0.10)

    for subplot_number, ticker in enumerate(TICKERS, start=1):
        row, col = positions[subplot_number - 1]
        pair = panels[ticker][['log_iv_daily', 'target_log_rv_tplus1']].
↳dropna().copy()
        x, y = pair['log_iv_daily'].to_numpy(), pair['target_log_rv_tplus1'].
↳to_numpy()
        coeffs = np.polyfit(x, y, 1)
        poly = np.poly1d(coeffs)
        x_line = np.linspace(x.min(), x.max(), 200)
        corr_value = pair['log_iv_daily'].corr(pair['target_log_rv_tplus1'])

        fig.add_trace(go.Scatter(x=x, y=y, mode='markers', name=ticker,
            marker=dict(color=AIRLINE_COLORS[ticker], size=6, opacity=0.55),
            showlegend=False,
            hovertemplate=f'{ticker}<br>log(IV_t)=%{{x:.
↳2f}}<br>log(RV_t+1)=%{{y:.2f}}<extra></extra>'),
            row=row, col=col)
        fig.add_trace(go.Scatter(x=x_line, y=poly(x_line), mode='lines',
↳name=f'{ticker} fit',
            line=dict(color='#24323D', width=2), showlegend=False,
↳hoverinfo='skip'),
            row=row, col=col)

        suffix = axis_suffix(subplot_number)
        fig.add_annotation(x=0.98, y=0.96, xanchor='right', yanchor='top',
            xref=f'x{suffix} domain', yref=f'y{suffix} domain',
            text=f'r = {corr_value:.2f}<br>Linear slope = {coeffs[0]:.2f}',
            showarrow=False, align='right', bgcolor='rgba(255,255,255,0.82)')

    fig.update_xaxes(title='Current log implied volatility (daily variance_
↳scale)')
    fig.update_yaxes(title='Next-day log realized variance')

```

```

    return style_figure(fig, 'Current Implied Volatility vs Next-Day Realized_
↪Volatility', height=760)

def make_monthly_driver_scatter_figure(driver_col, degree, title, x_title,
↪fit_color, agg='mean'):
    positions = [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2)]
    fig = make_subplots(rows=2, cols=3, subplot_titles=TICKERS,
↪vertical_spacing=0.14, horizontal_spacing=0.10)

    for subplot_number, ticker in enumerate(TICKERS, start=1):
        row, col = positions[subplot_number - 1]
        pair = build_monthly_driver_pair(panels[ticker], driver_col, agg=agg)
        x, y = pair[driver_col].to_numpy(), pair['target_next_month_log_rv'].
↪to_numpy()
        coeffs, poly, r2_value = polynomial_r2(x, y, degree)
        x_line = np.linspace(x.min(), x.max(), 200)
        corr_value = pair[driver_col].corr(pair['target_next_month_log_rv'])

        fig.add_trace(go.Scatter(x=x, y=y, mode='markers', name=ticker,
            marker=dict(color=AIRLINE_COLORS[ticker], size=8, opacity=0.72),
            showlegend=False,
            hovertemplate=f'{ticker}<br>{x_title}=%{x:.2f}<br>Next-month log_
↪RV=%{y:.2f}<extra></extra>'),
            row=row, col=col)
        fig.add_trace(go.Scatter(x=x_line, y=poly(x_line), mode='lines',
↪name=f'{ticker} fit',
            line=dict(color=fit_color, width=2), showlegend=False,
↪hoverinfo='skip'),
            row=row, col=col)

        suffix = axis_suffix(subplot_number)
        fig.add_annotation(x=0.98, y=0.96, xanchor='right', yanchor='top',
            xref=f'x{suffix} domain', yref=f'y{suffix} domain',
            text=f'r = {corr_value:.2f}<br>Degree {degree} R\u00b2 = {r2_value:.
↪2f}',
            showarrow=False, align='right', bgcolor='rgba(255,255,255,0.82)')

        fig.update_xaxes(title=x_title)
        fig.update_yaxes(title='Next-month average log realized variance')
    return style_figure(fig, title, height=760)

# --- Model results figures (OLS only for clarity) ---

def make_ols_sharpe_by_spec_figure():

```

```

"""2x2 grid â€" one panel per ticker. OLS straddle Sharpe by feature spec.
Best bar highlighted in airline color; others faded to 30%."""
positions = [(1, 1), (1, 2), (1, 3), (2, 1), (2, 2)]
spec_order = list(FEATURE_SPECS.keys())
x_labels = [SPEC_SHORT[s] for s in spec_order]

fig = make_subplots(
    rows=2, cols=3,
    subplot_titles=[f'{t} â€" OLS straddle Sharpe by feature spec' for t in
↪TICKERS],
    vertical_spacing=0.16, horizontal_spacing=0.12,
)

for idx, ticker in enumerate(TICKERS):
    row, col = positions[idx]
    ols_rows = results_df[
        (results_df['Ticker'] == ticker) & (results_df['Model_Family'] ==
↪'OLS')
    ].set_index('Feature_Spec').reindex(spec_order)

    sharpes = ols_rows['Sharpe_Straddle'].to_numpy(dtype=float)
    best_idx = int(np.nanargmax(sharpes))
    airline_color = AIRLINE_COLORS[ticker]

    bar_colors = [
        airline_color if i == best_idx else 'rgba(100,100,120,0.28)'
        for i in range(len(spec_order))
    ]
    border_widths = [2.0 if i == best_idx else 0.4 for i in
↪range(len(spec_order))]

fig.add_trace(go.Bar(
    x=x_labels,
    y=sharpes,
    name=ticker,
    showlegend=False,
    marker=dict(
        color=bar_colors,
        line=dict(color='#24323D', width=border_widths),
    ),
    customdata=ols_rows[['RMSE', 'Directional_Acc', 'Pct_Days_Traded']].
↪to_numpy(),
    hovertemplate=(
        f'{ticker} OLS<br>Spec={{x}}<br>Sharpe={{y:.3f}}<br>'
        'RMSE={{customdata[0]:.4f}}<br>Dir acc={{customdata[1]:.1%}}<br>'
        'Days traded={{customdata[2]:.1%}}<extra></extra>'
    ),
),

```

```

    ), row=row, col=col)

    best_spec = spec_order[best_idx]
    best_sharpe = float(sharpes[best_idx])
    suffix = axis_suffix(idx + 1)
    fig.add_annotation(
        x=0.98, y=0.97, xanchor='right', yanchor='top',
        xref=f'x{suffix} domain', yref=f'y{suffix} domain',
        text=f'Best: {SPEC_SHORT[best_spec]}<br>Sharpe = {best_sharpe:.2f}',
        showarrow=False, font=dict(size=10), bgcolor='rgba(255,255,255,0.
↪88)',
    )

    fig.update_yaxes(title='Straddle Sharpe (threshold-selected)')
    fig.update_xaxes(tickfont=dict(size=10))
    fig = style_figure(fig, 'OLS Straddle Sharpe by Feature Specification â€œ
↪Per Ticker', height=820)
    fig.update_layout(margin=dict(l=60, r=30, t=95, b=60))
    fig.add_annotation(
        x=0.5, y=1.04, xref='paper', yref='paper', xanchor='center',
        text='Highlighted bar = best Sharpe for that ticker Â· model family =
↪OLS only',
        showarrow=False, font=dict(size=11), bgcolor='rgba(255,255,255,0.85)',
    )
    return fig

def make_iv_lift_figure():
    """Grouped bars: HAR-RV, +IV, +IV+OVX+TOSI OLS Sharpe, one group per ticker.
    JETS annotated with DM significance."""
    lift_specs = ['HAR-RV', 'HAR-RV+IV', 'HAR-RV+IV+OVX+TOSI']
    lift_colors = ['#8C8C8C', '#3C91E6', '#2D6A4F']
    lift_labels = ['HAR-RV', '+IV', '+IV+OVX+TOSI']

    fig = go.Figure()

    for spec, color, label in zip(lift_specs, lift_colors, lift_labels):
        y_vals, customdata_rows = [], []
        for ticker in TICKERS:
            row = results_df[
                (results_df['Ticker'] == ticker) &
                (results_df['Feature_Spec'] == spec) &
                (results_df['Model_Family'] == 'OLS')
            ]
            if row.empty:
                y_vals.append(None)
                customdata_rows.append([None, None])

```

```

        else:
            r = row.iloc[0]
            y_vals.append(r['Sharpe_Straddle'])
            customdata_rows.append([r['RMSE'], r['Directional_Acc']])

    fig.add_trace(go.Bar(
        x=TICKERS,
        y=y_vals,
        name=label,
        marker_color=color,
        customdata=customdata_rows,
        hovertemplate=(
            f'Spec: {label}<br>Ticker=%{{x}}<br>Sharpe=%{{y:.3f}}<br>'
            'RMSE=%{{customdata[0]:.4f}}<br>Dir acc=%{{customdata[1]:.
↪1%}}<extra></extra>'
        ),
    ))

    jets_iv_dm = dm_results_df[
        (dm_results_df['Ticker'] == 'JETS') &
        (dm_results_df['Hypothesis'] == 'IV adds to HAR-RV?')
    ]
    if not jets_iv_dm.empty:
        pval = jets_iv_dm.iloc[0]['P_Value']
        jets_ols = results_df[
            (results_df['Ticker'] == 'JETS') & (results_df['Model_Family'] == '
↪OLS')
        ]['Sharpe_Straddle']
        y_anchor = max(jets_ols.max() if not jets_ols.empty else 0, 0) + 0.25
        fig.add_annotation(
            x='JETS', y=y_anchor,
            text=f'DM p={pval:.4f} \u2713',
            showarrow=True, arrowhead=2,
            font=dict(size=11, color='#2D6A4F'),
            bgcolor='rgba(255,255,255,0.88)',
        )

    fig.update_xaxes(title='Ticker')
    fig.update_yaxes(title='OLS straddle Sharpe (threshold-selected)')
    fig.update_layout(barmode='group',
        legend=dict(orientation='h', y=-0.14, x=0.5, xanchor='center',
↪font=dict(size=12)))
    fig = style_figure(fig, 'IV Adds Sharpe for JETS â€" HAR-RV vs +IV vs
↪+IV+OVX+TOSI (OLS)', height=580)
    fig.update_layout(margin=dict(l=60, r=30, t=80, b=100))
    fig.add_annotation(
        x=0.99, y=1.08, xref='paper', yref='paper', xanchor='right',

```



```

        text='\u2713 = DM test significant at 5% \u00c5 model family = OLS only',
        showarrow=False, font=dict(size=10), bgcolor='rgba(255,255,255,0.85)',
    )
    return fig

def make_dm_heatmap_figure():
    """Heatmap of log10(p-value) for all DM tests.
    Rows = hypothesis (short label), cols = tickers."""
    hyp_order = [
        'OVX adds to HAR-RV?',
        'TOSI adds to HAR-RV+OVX?',
        'IV adds to HAR-RV?',
        'OVX adds to HAR-RV+IV?',
        'TOSI adds to HAR-RV+IV+OVX?',
        'IV adds to HAR-RV+OVX?',
    ]
    hyp_short = {
        'OVX adds to HAR-RV?': 'OVX adds to HAR-RV?',
        'TOSI adds to HAR-RV+OVX?': 'TOSI adds to +OVX?',
        'IV adds to HAR-RV?': 'IV adds to HAR-RV?',
        'OVX adds to HAR-RV+IV?': 'OVX adds to +IV?',
        'TOSI adds to HAR-RV+IV+OVX?': 'TOSI adds to +IV+OVX?',
        'IV adds to HAR-RV+OVX?': 'IV adds to +OVX?',
    }
    z_vals, text_vals = [], []
    for hyp in hyp_order:
        row_z, row_t = [], []
        for ticker in TICKERS:
            match = dm_results_df[
                (dm_results_df['Ticker'] == ticker) &
                (dm_results_df['Hypothesis'] == hyp)
            ]
            if match.empty:
                row_z.append(0.0)
                row_t.append('n/a')
            else:
                pval = float(match.iloc[0]['P_Value'])
                dm_stat = float(match.iloc[0]['DM_Stat'])
                neg_log_p = -np.log10(max(pval, 1e-6))
                row_z.append(neg_log_p)
                sig_str = '\u2713' if pval < 0.05 else ''
                better = match.iloc[0]['Better_Model']
                row_t.append(f'p={pval:.3f}{sig_str}<br>DM={dm_stat:+.
                2f}<br>{better}')
            z_vals.append(row_z)
            text_vals.append(row_t)

```

```

y_labels = [hyp_short[h] for h in hyp_order]
sig_threshold = -np.log10(0.05)

fig = go.Figure(go.Heatmap(
    z=z_vals,
    x=TICKERS,
    y=y_labels,
    colorscale=[
        [0.0, 'rgba(230,230,230,0.5)'],
        [0.3, 'rgba(180,210,180,0.7)'],
        [1.0, '#2D6A4F'],
    ],
    zmin=0,
    zmax=4,
    text=text_vals,
    texttemplate='%{text}',
    hovertemplate='Ticker=%{x}<br>Hypothesis=%{y}<br>-log10(p)=%{z:.  

    2f}<extra></extra>',
    colorbar=dict(
        title='-log10(p)',
        tickvals=[0, sig_threshold, 2, 3, 4],
        ticktext=['0', '5% threshold', 'p=0.01', 'p=0.001', 'p=0.0001'],
    ),
))

fig = style_figure(fig, 'Diebold-Mariano Test Significance &" All Tickers  

and Hypotheses', height=500)
fig.update_layout(margin=dict(l=170, r=90, t=80, b=50))
fig.add_annotation(
    x=0.5, y=1.10, xref='paper', yref='paper', xanchor='center',
    text='Green = extended model significantly better  $\hat{u}_{2713} = p < 0.05$   

 $\hat{u}_{2713}$  Only JETS IV tests are significant',
    showarrow=False, font=dict(size=11), bgcolor='rgba(255,255,255,0.85)',
)
return fig

def make_jets_forecast_figure():
    """JETS actual VRP vs HAR-RV OLS vs best OLS spec, with RMSE and DM  

    annotations."""
    specs_to_show = {
        'HAR-RV': dict(color='#9E9E9E', dash='dash', width=2.0),
        'HAR-RV+IV+OVX+TOSI': dict(color=AIRLINE_COLORS['JETS'], dash='solid',  

width=2.5),
    }

```

```

jets_preds = predictions_df[
    (predictions_df['Ticker'] == 'JETS') & (predictions_df['Model_Family'] ==
    ↪ 'OLS')
].copy()

actual = (
    jets_preds[jets_preds['Feature_Spec'] == 'HAR-RV'][['trade_date',
    ↪ 'vrp_actual']]
    .drop_duplicates('trade_date').sort_values('trade_date')
)

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=actual['trade_date'], y=actual['vrp_actual'],
    mode='lines', name='Actual VRP',
    line=dict(color='#24323D', width=1.8),
    hovertemplate='Actual VRP<br>{%x|%Y-%m-%d}<br>{%y:.3f}<extra></extra>',
))

for spec, style in specs_to_show.items():
    pred_df = jets_preds[jets_preds['Feature_Spec'] == spec].
    ↪sort_values('trade_date')
    metrics_row = results_df[
        (results_df['Ticker'] == 'JETS') &
        (results_df['Feature_Spec'] == spec) &
        (results_df['Model_Family'] == 'OLS')
    ]
    rmse_val = metrics_row.iloc[0]['RMSE'] if not metrics_row.empty else
    ↪float('nan')
    sharpe_val = metrics_row.iloc[0]['Sharpe_Straddle'] if not metrics_row.
    ↪empty else float('nan')
    label = f'{SPEC_SHORT[spec]} (RMSE={rmse_val:.4f}, Sharpe={sharpe_val:.
    ↪2f})'

    fig.add_trace(go.Scatter(
        x=pred_df['trade_date'], y=pred_df['y_pred_vrp'],
        mode='lines', name=label,
        line=dict(color=style['color'], dash=style['dash'],
    ↪width=style['width']),
        hovertemplate=f'{SPEC_SHORT[spec]}<br>{%x|%Y-%m-%d}<br>{%y:.
    ↪3f}<extra></extra>',
    ))

    jets_iv_dm = dm_results_df[
        (dm_results_df['Ticker'] == 'JETS') & (dm_results_df['Hypothesis'] ==
    ↪ 'IV adds to HAR-RV?')

```

```

]
if not jets_iv_dm.empty:
    pval = jets_iv_dm.iloc[0]['P_Value']
    dm_stat = jets_iv_dm.iloc[0]['DM_Stat']
    fig.add_annotation(
        x=0.01, y=0.97, xref='paper', yref='paper', xanchor='left',
        yanchor='top',
        text=f'DM test (IV vs HAR-RV): stat={dm_stat:.2f}, p={pval:.4f} â€"
        significant at 1%',
        showarrow=False, font=dict(size=11, color='#2D6A4F'),
        bgcolor='rgba(255,255,255,0.88)',
    )

    add_shaded_periods(fig)
    fig.update_xaxes(title='Walk-forward OOS date', rangeslider_visible=False)
    fig.update_yaxes(title='VRP (log-variance scale)')
    fig.update_layout(
        legend=dict(orientation='h', y=-0.16, x=0.5, xanchor='center',
        font=dict(size=11)),
    )
    fig = style_figure(fig, 'JETS OOS VRP Forecast â€" HAR-RV vs +IV+OVX+TOSI
    (OLS)', height=560)
    fig.update_layout(margin=dict(l=60, r=30, t=80, b=110))
    return fig

def make_jets_cumulative_pnl_figure():
    """Cumulative straddle P&L for JETS â€" HAR-RV OLS vs +IV+OVX+TOSI OLS."""
    SQRT2_PI_local = np.sqrt(2 / np.pi)
    specs_to_show = {
        'HAR-RV': dict(color='#9E9E9E', dash='dash', width=2.0,
        label='HAR-RV'),
        'HAR-RV+IV+OVX+TOSI': dict(color=AIRLINE_COLORS['JETS'], dash='solid',
        width=2.5, label='+IV+OVX+TOSI'),
    }

    fig = go.Figure()

    for spec, style in specs_to_show.items():
        pred_df = predictions_df[
            (predictions_df['Ticker'] == 'JETS') &
            (predictions_df['Feature_Spec'] == spec) &
            (predictions_df['Model_Family'] == 'OLS')
        ].sort_values('trade_date').copy()

```

```

    pnl = pred_df['signal'] * (pred_df['abs_daily_return_tplus1'] -
↳Sqrt2_PI_local * pred_df['iv_daily_vol'])
    cum_pnl = pnl.cumsum()

    metrics_row = results_df[
        (results_df['Ticker'] == 'JETS') &
        (results_df['Feature_Spec'] == spec) &
        (results_df['Model_Family'] == 'OLS')
    ]
    sharpe_val = metrics_row.iloc[0]['Sharpe_Straddle'] if not metrics_row.
↳empty else float('nan')
    pct_traded = metrics_row.iloc[0]['Pct_Days_Traded'] if not metrics_row.
↳empty else float('nan')
    trace_label = f'{style["label"]} (Sharpe={sharpe_val:.2f}, {pct_traded:.
↳0%} traded)'

    fig.add_trace(go.Scatter(
        x=pred_df['trade_date'],
        y=cum_pnl,
        mode='lines',
        name=trace_label,
        line=dict(color=style['color'], dash=style['dash'],
↳width=style['width']),
        hovertemplate=f'{style["label"]} <br> %{{x|%Y-%m-%d}} <br> Cum P&L= %{{y:
↳.4f}} <extra></extra>',
    ))

    fig.add_hline(y=0, line_dash='dot', line_color='#7F8C8D', line_width=1)
    add_shaded_periods(fig)
    fig.update_xaxes(title='Walk-forward OOS date')
    fig.update_yaxes(title='Cumulative straddle P&L (log-return units)')
    fig.update_layout(
        legend=dict(orientation='h', y=-0.16, x=0.5, xanchor='center',
↳font=dict(size=11)),
    )
    fig = style_figure(fig, 'JETS Cumulative ATM Straddle P&L â€" HAR-RV vs
↳IV+OVX+TOSI (OLS)', height=520)
    fig.update_layout(margin=dict(l=60, r=30, t=80, b=110))
    fig.add_annotation(
        x=0.99, y=0.03, xref='paper', yref='paper', xanchor='right',
↳yanchor='bottom',
        text='P&L = signal x (|daily return| - sqrt(2/pi) x daily IV vol)',
        showarrow=False, font=dict(size=10), bgcolor='rgba(255,255,255,0.85)',
    )
    return fig

```

```

def make_feature_importance_figure(ticker='JETS', top_n=10):
    fig = make_subplots(rows=1, cols=2, subplot_titles=['XGBoost', 'Random_
↳Forest'], horizontal_spacing=0.26)
    for col_index, family_name in enumerate(['XGBoost', 'Random Forest'],
↳start=1):
        importance = feature_importances[ticker][family_name]
        importance = importance.head(top_n).sort_values(ascending=True)
        fig.add_trace(go.Bar(
            x=importance.values, y=importance.index,
            orientation='h', name=family_name, showlegend=False,
            marker_color=MODEL_COLORS[family_name],
            hovertemplate='%{y}<br>Importance=%{x:.3f}<extra></extra>',
            row=1, col=col_index)

        fig.update_xaxes(title='Feature importance')
        fig = style_figure(fig, f'Top {top_n} Tree-Based Drivers for {ticker} VRP_
↳Under HAR-RV+IV+OVX+TOSI', height=650)
        fig.update_layout(margin=dict(l=180, r=30, t=80, b=55))
    return fig

# --- Build all figures ---
fig_realized_vol = make_sector_realized_vol_figure()
fig_oil_drivers = make_oil_driver_figure()
fig_correlation = make_correlation_heatmap()
fig_iv_scatter = make_iv_scatter_figure()
fig_ovx_scatter = make_monthly_driver_scatter_figure(
    driver_col='log_ovx_daily', degree=3,
    title='OVX vs Next-Month Realized Volatility With Cubic Fits',
    x_title='Current-month average log OVX (daily variance scale)',
    fit_color='#C75146', agg='mean',
)
fig_tosi_scatter = make_monthly_driver_scatter_figure(
    driver_col='tosi_level', degree=1,
    title='TOSI vs Next-Month Realized Volatility With Linear Fits',
    x_title='Current-month TOSI', fit_color='#3C91E6', agg='last',
)
fig_ols_sharpe = make_ols_sharpe_by_spec_figure()
fig_iv_lift = make_iv_lift_figure()
fig_dm_heatmap = make_dm_heatmap_figure()
fig_jets_forecast = make_jets_forecast_figure()
fig_jets_pnl = make_jets_cumulative_pnl_figure()
fig_feature_importance = make_feature_importance_figure(ticker='JETS')

```

## 1.8 6. Results Summary

Display the cleaned data inventory, the main evaluation tables, and the final interactive figures selected to summarize both the data relationships and the model results.

```
[ ]: import json
import pickle
print('Data inventory')
display(data_inventory)

print('Cleaning log')
display(cleaning_log)

print('HAR feature definitions')
display(feature_map)

print('Average metrics by feature specification and model family')
display(summary_df)

print('Best model by ticker (lowest RMSE)')
display(best_models_df[['Ticker', 'Model', 'RMSE', 'Sharpe_Straddle',
    ↳ 'Directional_Acc']])

results_snapshot_df = results_df[[
    'Ticker', 'Feature_Spec', 'Model_Family', 'Model', 'RMSE', 'MAE', 'R2',
    'Directional_Acc', 'VRP_Pos_Base_Rate', 'Pct_Days_Traded', 'Avg_Threshold',
    ↳ 'Avg_Pct_Traded',
    'Sharpe_Straddle', 'Mean_Straddle_PnL', 'Signal_Hit_Rate',
    'RMSE_vs_OLS_HAR_RV', 'Sharpe_vs_OLS_HAR_RV', 'N_Folds', 'N_Predictions'
]].sort_values(['Ticker', 'RMSE', 'Sharpe_Straddle'], ascending=[True, True,
    ↳ False]).reset_index(drop=True)

print('Expanded per-ticker metric snapshot')
display(results_snapshot_df)

print('\nDiebold-Mariano significance tests (OLS models, H0: equal MSE,
    ↳ accuracy)')
display(dm_results_df)

# ----- Artifact export for Streamlit dashboard -----
processed_dir.mkdir(parents=True, exist_ok=True)

# Tall per-ticker feature panel (one parquet, ticker column)
panels_tall = pd.concat(
    [p.assign(Ticker=t) for t, p in panels.items()], ignore_index=True
)

feature_importance_rows = []
```

```

for ticker, fam_map in feature_importances.items():
    for family, series in (fam_map or {}).items():
        if series is None:
            continue
        for feat, val in series.items():
            feature_importance_rows.append({
                'Ticker': ticker, 'Model_Family': family,
                'Feature': feat, 'Importance': float(val),
            })
feature_importance_df = pd.DataFrame(feature_importance_rows)

# Macro / sentiment series as long tables
ovx_df = ovx.reset_index().rename(columns={'Date': 'Date', 'OVX': 'OVX'})
tosi_df_out = tosi_level.reset_index().rename(columns={'Date': 'Date', 'TOSI': 'TOSI'})

iv_frames = []
for t, s in iv_map.items():
    frame = s.reset_index().rename(columns={'Date': 'Date', 'IV': 'IV'})
    frame['Ticker'] = t
    iv_frames.append(frame)
iv_long_df = pd.concat(iv_frames, ignore_index=True)

# Feature-spec catalog
feature_specs_df = pd.DataFrame([
    {'Feature_Spec': k, 'Features': ','.join(v), 'Feature_Count': len(v)}
    for k, v in FEATURE_SPECS.items()
])

artifact_tables = {
    'rv_panel': rv_panel,
    'panels': panels_tall,
    'monthly_exploration': monthly_exploration.reset_index(),
    'data_inventory': data_inventory,
    'cleaning_log': cleaning_log,
    'feature_map': feature_map,
    'feature_specs': feature_specs_df,
    'results': results_df,
    'results_snapshot': results_snapshot_df,
    'summary': summary_df,
    'family_summary': family_summary_df,
    'ridge_vs_ols': ridge_vs_ols_df,
    'best_models': best_models_df,
    'predictions': predictions_df,
    'dm_results': dm_results_df,
    'threshold_summary': threshold_summary_df,
    'threshold_log_all': threshold_log_all_df,
    'feature_importances': feature_importance_df,
}

```



```

    'ovx': ovx_df,
    'tosi': tosi_df_out,
    'iv_long': iv_long_df,
    'visual_catalog': visual_catalog,
}

for name, df in artifact_tables.items():
    out = processed_dir / f'{name}.parquet'
    try:
        df.to_parquet(out, index=False)
    except Exception:
        df.to_csv(processed_dir / f'{name}.csv', index=False)

# Save Plotly figures as JSON so the Streamlit app can render them without
↳ rebuilding.
figure_registry = {
    'realized_vol': fig_realized_vol,
    'oil_drivers': fig_oil_drivers,
    'correlation': fig_correlation,
    'iv_scatter': fig_iv_scatter,
    'ovx_scatter': fig_ovx_scatter,
    'tosi_scatter': fig_tosi_scatter,
    'ols_sharpe': fig_ols_sharpe,
    'iv_lift': fig_iv_lift,
    'dm_heatmap': fig_dm_heatmap,
    'jets_forecast': fig_jets_forecast,
    'jets_pnl': fig_jets_pnl,
    'feature_importance_jets': fig_feature_importance,
}
figures_dir = processed_dir / 'figures'
figures_dir.mkdir(parents=True, exist_ok=True)
for name, fig in figure_registry.items():
    (figures_dir / f'{name}.json').write_text(pio.to_json(fig),
    encoding='utf-8')

# Palette + constants used by the Streamlit app
meta = {
    'tickers': TICKERS,
    'model_families': MODEL_FAMILIES,
    'feature_specs': {k: v for k, v in FEATURE_SPECS.items()},
    'spec_short': SPEC_SHORT,
    'airline_colors': AIRLINE_COLORS,
    'model_colors': MODEL_COLORS,
    'spec_colors': SPEC_COLORS,
    'shaded_periods': SHADED_PERIODS,
    'sqrt2_pi': float(SQRT2_PI),
    'trade_frac_warn': TRADE_FRAC_WARN,

```

```

        'oos_start': str(predictions_df['trade_date'].min().date()),
        'oos_end': str(predictions_df['trade_date'].max().date()),
    }
    (processed_dir / 'meta.json').write_text(json.dumps(meta, indent=2,
↪default=str), encoding='utf-8')

    print('\nSaved artifacts to', processed_dir)
    for name in sorted(artifact_tables):
        print(f' - {name}.parquet')
    print(f' - {len(figure_registry)} plotly figures under figures/')
    print(' - meta.json')

    # Re-display figures
    for fig in [fig_realized_vol, fig_oil_drivers, fig_correlation,
                fig_iv_scatter, fig_ovx_scatter, fig_tosi_scatter,
                fig_ols_sharpe, fig_iv_lift, fig_dm_heatmap,
                fig_jets_forecast, fig_jets_pnl, fig_feature_importance]:
        fig.show()

```